



Universidad Nacional Autónoma de México
Facultad de Ciencias



Licenciatura en Ciencias de la Computación

Seminario de Ciencias de la Computación B

Computación en la nube

Tarea 4

Programación del servidor Zuul de Netflix.

Elizalde Maza Jesús Eduardo
321031686
Peredo López Citlalli Abigail
321161022

Profesor Gustavo Arturo Marquez Flores

30 de abril del 2026

Objetivo

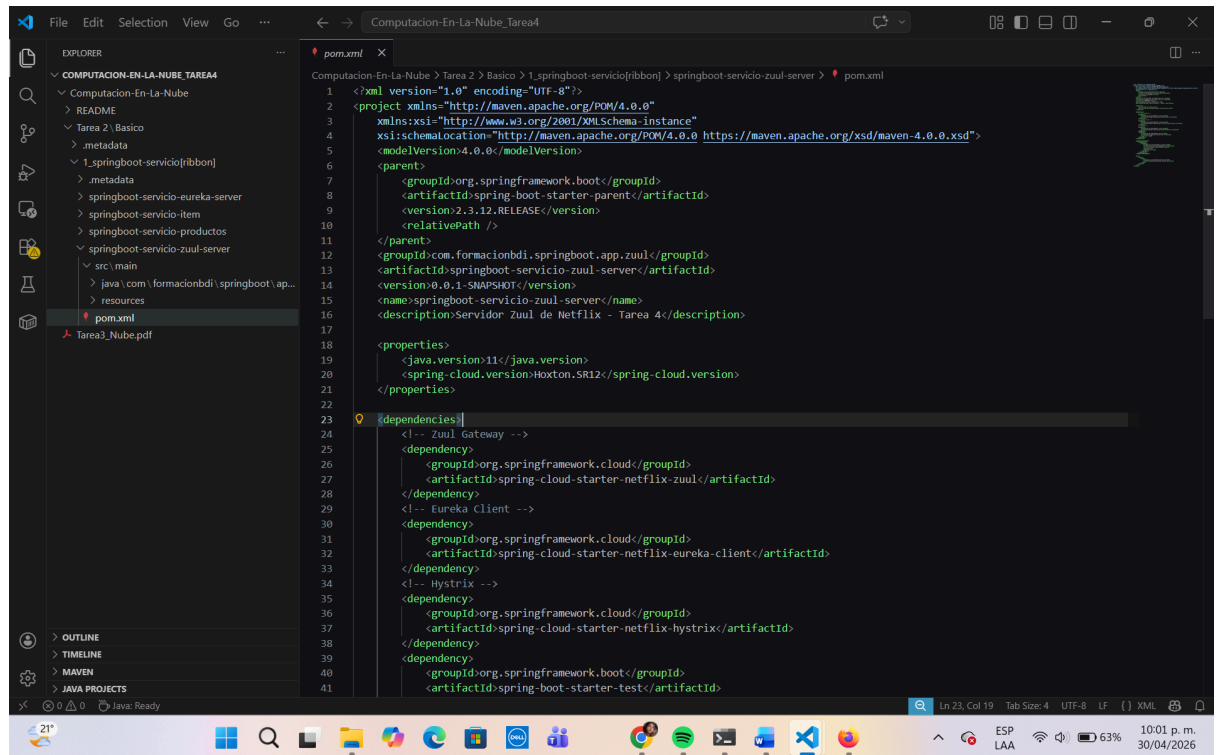
Instalación, configuración y funcionalidades del servidor Zuul de Netflix.

Especificación

Continuación de la tarea 3. Programar en la tarea 3 las siguientes actividades:

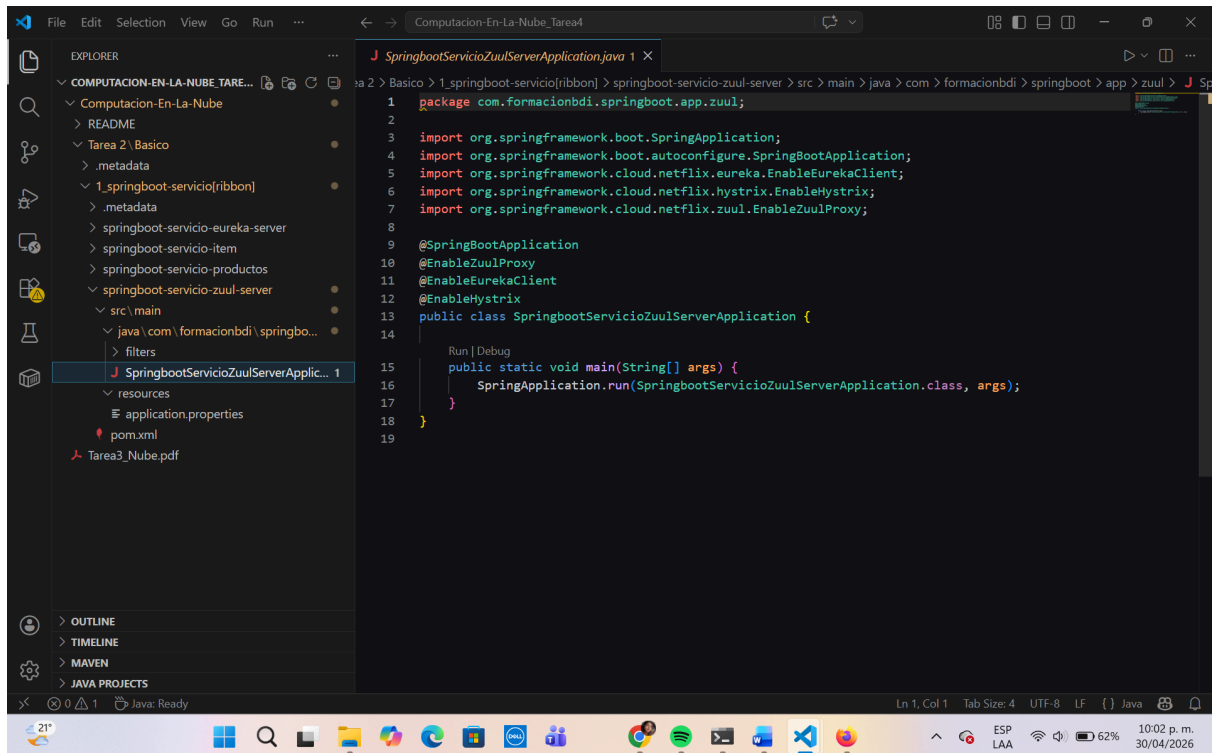
- Instalar y configurar el servidor de Zuul.

Dependencias Maven (pom.xml)

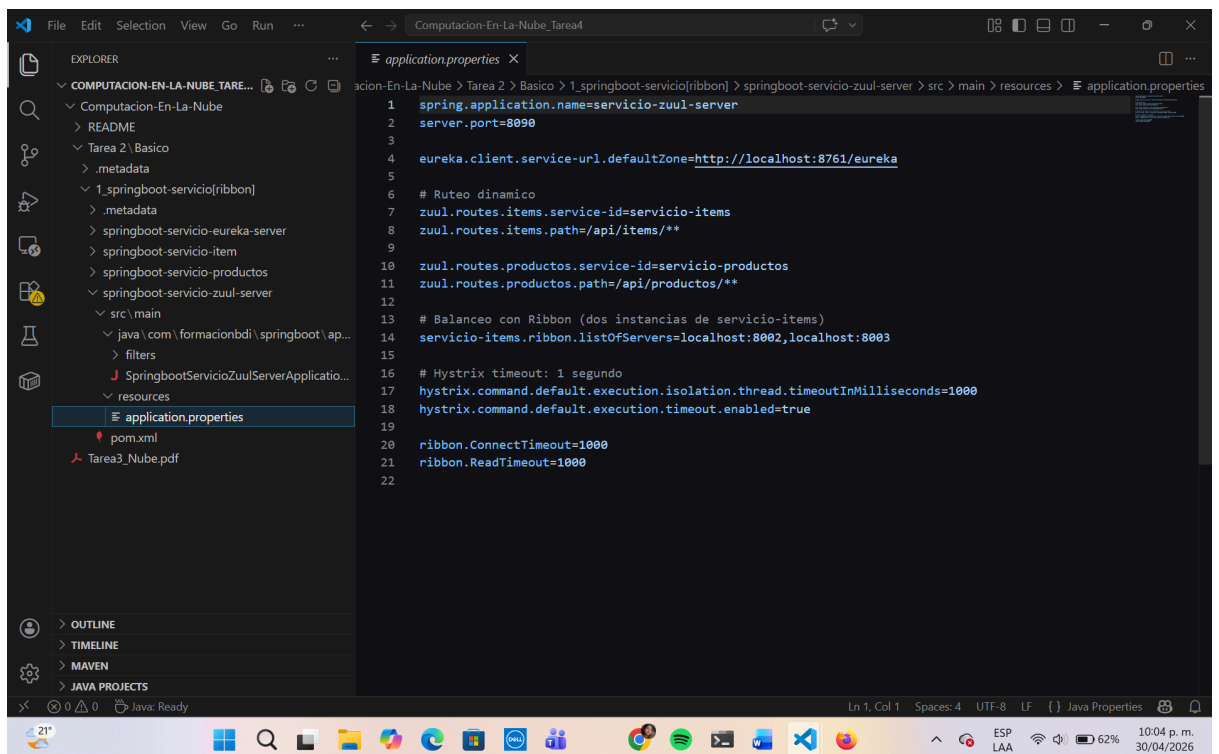


```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0"
3   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4   xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
5   <modelVersion>4.0.0</modelVersion>
6   <parent>
7     <groupId>org.springframework.boot</groupId>
8     <artifactId>spring-boot-starter-parent</artifactId>
9     <version>2.3.12.RELEASE</version>
10    <relativePath>/>
11  </parent>
12  <groupId>com.formacionbdi.springboot.app.zuul</groupId>
13  <artifactId>springboot-servicio-zuul-server</artifactId>
14  <version>0.0.1-SNAPSHOT</version>
15  <name>springboot-servicio-zuul-server</name>
16  <description>Servidor zuul de Netflix - Tarea 4</description>
17
18  <properties>
19    <java.version>11</java.version>
20    <spring-cloud.version>Hoxton.SR12</spring-cloud.version>
21  </properties>
22
23  <dependencies>
24    <!-- Zuul Gateway -->
25    <dependency>
26      <groupId>org.springframework.cloud</groupId>
27      <artifactId>spring-cloud-starter-netflix-zuul</artifactId>
28    </dependency>
29    <!-- Eureka Client -->
30    <dependency>
31      <groupId>org.springframework.cloud</groupId>
32      <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
33    </dependency>
34    <!-- Hystrix -->
35    <dependency>
36      <groupId>org.springframework.cloud</groupId>
37      <artifactId>spring-cloud-starter-netflix-hystrix</artifactId>
38    </dependency>
39    <dependency>
40      <groupId>org.springframework.boot</groupId>
41      <artifactId>spring-boot-starter-test</artifactId>
```

Clase principal



Configuración (application.properties)



URL: <http://localhost:8090/api/productos/listar>

Zuul recibe la petición en el puerto 8090 y la redirige transparentemente al servicio-productos en el puerto 8001. La respuesta contiene el catálogo de productos registrado en la base de datos H2.

localhost:8090/api/productos/listar

Dar formato al texto

```
[{"id":1,"marca":"Ford","modelo":"Focus","anio":2010,"precio":250000.0,"createAt":"2026-04-30"},
{"id":2,"marca":"Ford","modelo":"Fusion","anio":2016,"precio":350000.0,"createAt":"2026-04-30"},
{"id":3,"marca":"GMC","modelo":"Acadia","anio":2026,"precio":1352000.0,"createAt":"2026-04-30"},
{"id":4,"marca":"GMC","modelo":"Acadia","anio":2026,"precio":1352000.0,"createAt":"2026-04-30"},
{"id":5,"marca":"GMC","modelo":"Terrain","anio":2026,"precio":802000.0,"createAt":"2026-04-30"},
{"id":6,"marca":"Ford","modelo":"Escape Híbrida","anio":2025,"precio":801100.0,"createAt":"2026-04-30"},
{"id":7,"marca":"Ford","modelo":"Territory Híbrida","anio":2025,"precio":719900.0,"createAt":"2026-04-30"},
{"id":8,"marca":"Toyota","modelo":"Corolla Cross HEV","anio":2024,"precio":625900.0,"createAt":"2026-04-30"},
{"id":9,"marca":"Toyota","modelo":"Highlander HEV","anio":2024,"precio":950900.0,"createAt":"2026-04-30"}]
```

- Programar las siguientes funcionalidades del servidor de Zuul:
 - Ruteo dinámico.

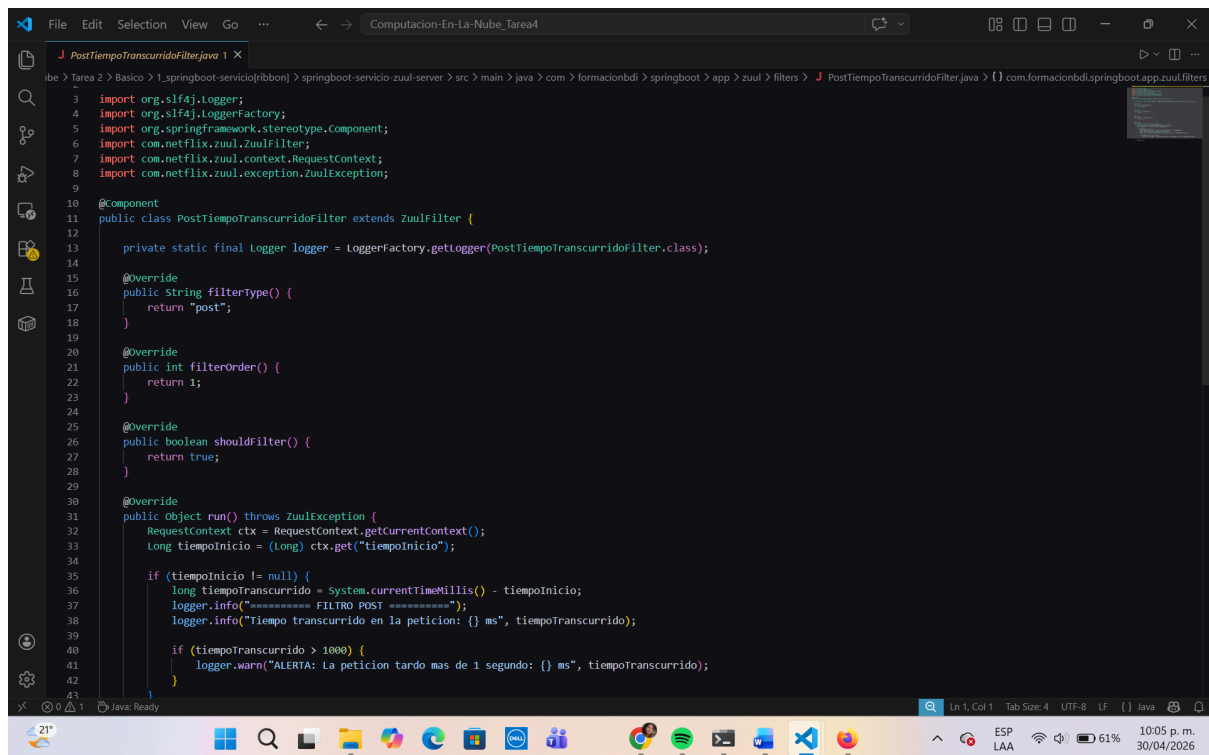
```
File Edit Selection View Go ... Computation-En-La-Nube_Tarea4

J PreTiempoTranscurridoFilter.java 1 x
ube > Tarea 2 > Básico > 1.springboot-servicio[ribbon] > springboot-servicio-zuul-server > src > main > java > com > formacionbdi > springboot > app > zuul > filters > J PreTiempoTranscurridoFilter.java > {} com.formacionbdi.springboot.app.zuul.filters

2
3 import javax.servlet.http.HttpServletRequest;
4 import org.slf4j.Logger;
5 import org.slf4j.LoggerFactory;
6 import org.springframework.stereotype.Component;
7 import com.netflix.zuul.ZuulFilter;
8 import com.netflix.zuul.context.RequestContext;
9 import com.netflix.zuul.exception.ZuulException;
10
11 @Component
12 public class PreTiempoTranscurridoFilter extends ZuulFilter {
13
14
15
16     private static final Logger logger = LoggerFactory.getLogger(PreTiempoTranscurridoFilter.class);
17
18     @Override
19     public String filterType() {
20         return "pre";
21     }
22
23     @Override
24     public int filterOrder() {
25         return 1;
26     }
27
28     @Override
29     public boolean shouldFilter() {
30         return true;
31     }
32
33     @Override
34     public Object run() throws ZuulException {
35         RequestContext ctx = RequestContext.getCurrentContext();
36         HttpServletRequest request = ctx.getRequest();
37
38         logger.info("===== FILTRO PRE =====");
39         logger.info("Entrando a la petición: {} {}", request.getMethod(), request.getRequestURL().toString());
40
41         ctx.set("tiempoInicio", System.currentTimeMillis());
42         return null;
43     }
44 }
```

Ln 1, Col 1 Tab Size: 4 UTF-8 LF Java

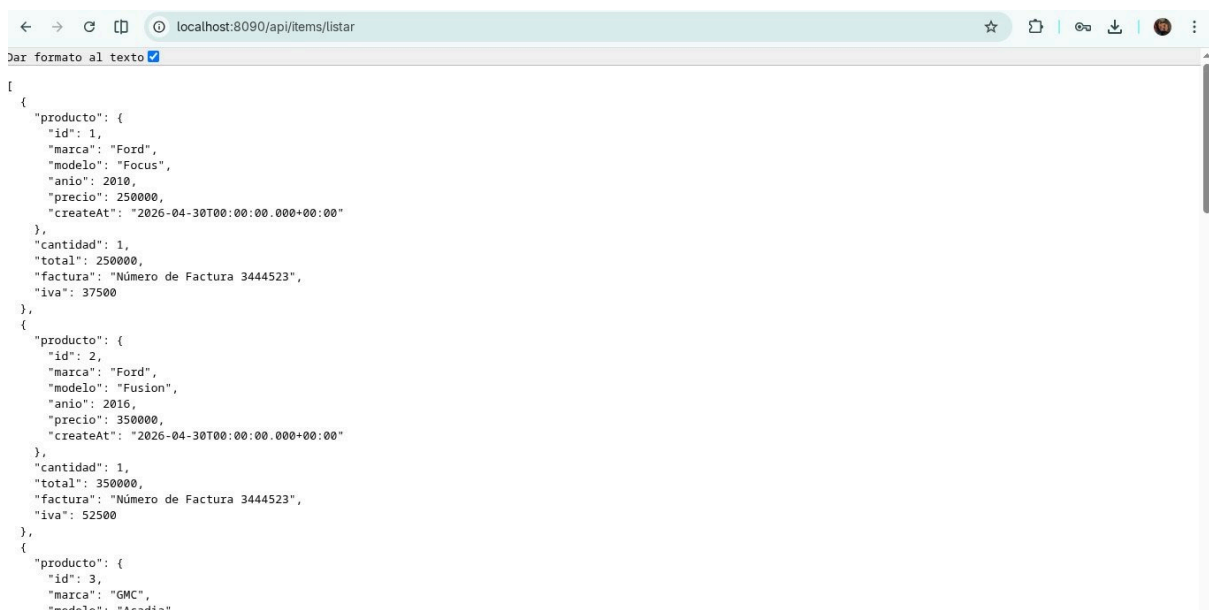
21° ESP LAA 61% 10:05 p. m. 30/04/2026



```
1 import org.slf4j.Logger;
2 import org.slf4j.LoggerFactory;
3 import org.springframework.stereotype.Component;
4 import com.netflix.zuul.ZuulFilter;
5 import com.netflix.zuul.context.RequestContext;
6 import com.netflix.zuul.exception.ZuulException;
7
8 @Component
9 public class PostTiempoTranscurridoFilter extends ZuulFilter {
10
11     private static final Logger logger = LoggerFactory.getLogger(PostTiempoTranscurridoFilter.class);
12
13     @Override
14     public String filterType() {
15         return "post";
16     }
17
18     @Override
19     public int filterOrder() {
20         return 1;
21     }
22
23     @Override
24     public boolean shouldFilter() {
25         return true;
26     }
27
28     @Override
29     public Object run() throws ZuulException {
30         RequestContext ctx = RequestContext.getCurrentContext();
31         Long tiempoInicio = (Long) ctx.get("tiempoInicio");
32
33         if (tiempoInicio != null) {
34             long tiempoTranscurrido = System.currentTimeMillis() - tiempoInicio;
35             logger.info("===== FILTRO POST =====");
36             logger.info("Tiempo transcurrido en la petición: {} ms", tiempoTranscurrido);
37
38             if (tiempoTranscurrido > 1000) {
39                 logger.warn("ALERTA: La petición tardó más de 1 segundo: {} ms", tiempoTranscurrido);
40             }
41         }
42     }
43 }
```

URL: <http://localhost:8090/api/items/listar>

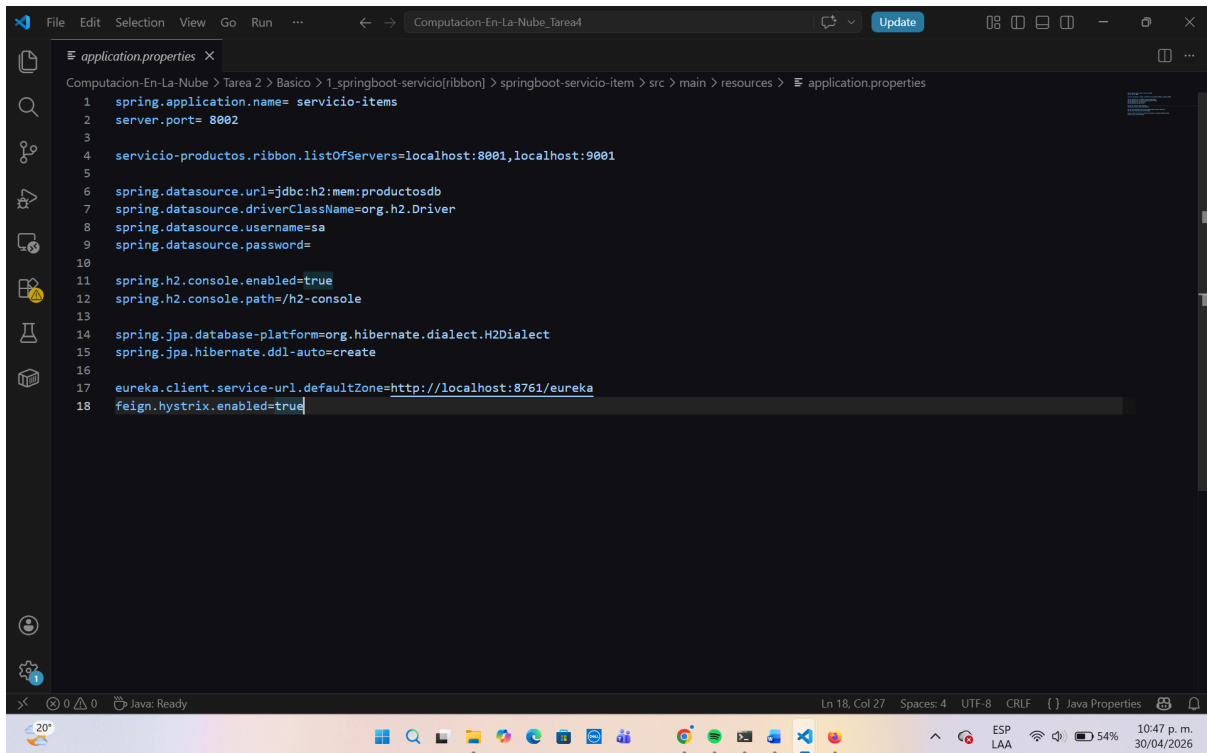
Zuul redirige la petición al servicio-item en el puerto 8002. El servicio-item consume internamente al servicio-productos usando Feign y devuelve los items enriquecidos con información de factura, total e IVA.



```
[
  {
    "producto": {
      "id": 1,
      "marca": "Ford",
      "modelo": "Focus",
      "año": 2010,
      "precio": 250000,
      "createdAt": "2026-04-30T00:00:00.000+00:00"
    },
    "cantidad": 1,
    "total": 250000,
    "factura": "Número de Factura 3444523",
    "iva": 37500
  },
  {
    "producto": {
      "id": 2,
      "marca": "Ford",
      "modelo": "Fusion",
      "año": 2016,
      "precio": 350000,
      "createdAt": "2026-04-30T00:00:00.000+00:00"
    },
    "cantidad": 1,
    "total": 350000,
    "factura": "Número de Factura 3444523",
    "iva": 52500
  },
  {
    "producto": {
      "id": 3,
      "marca": "GMC",
      "modelo": "Acadia",

```

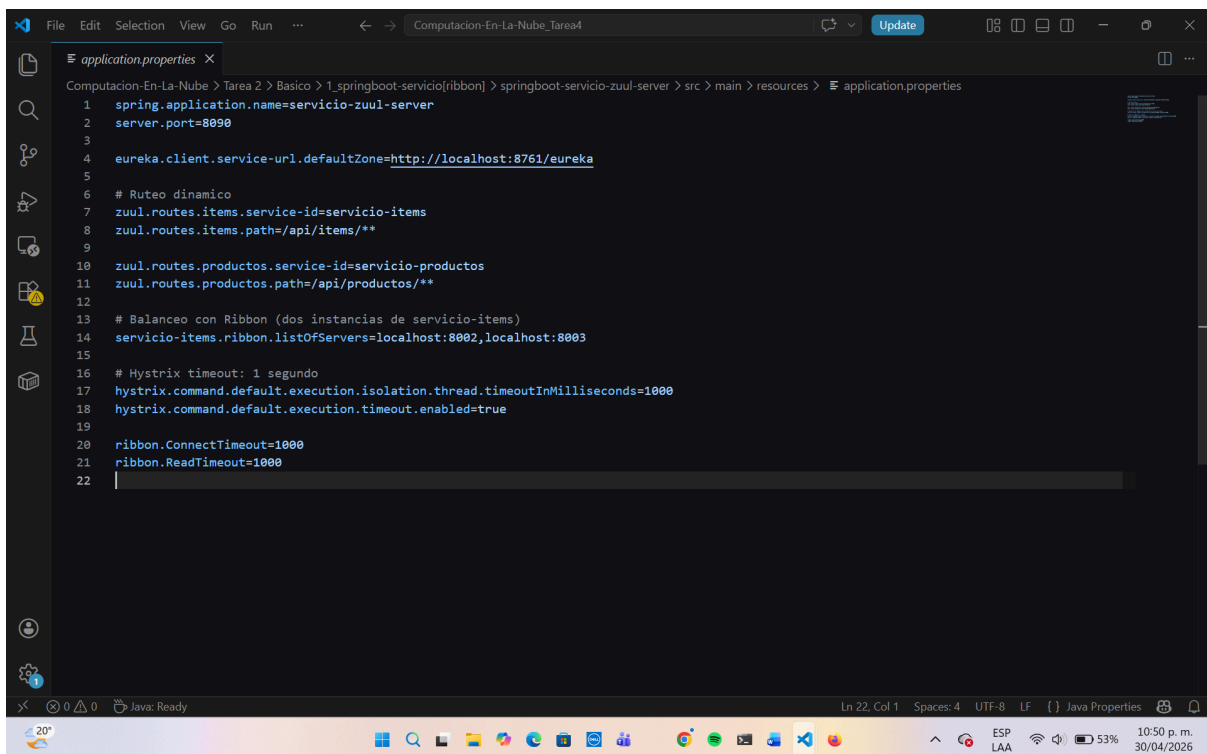
- Balanceo de puertos con Hystrix



The screenshot shows an IDE window titled 'Computation-En-La-Nube_Tarea4'. The editor displays the 'application.properties' file for the project 'springboot-servicio-items'. The properties are as follows:

```
1 spring.application.name= servicio-items
2 server.port= 8002
3
4 servicio-productos.ribbon.listOfServers=localhost:8001,localhost:9001
5
6 spring.datasource.url=jdbc:h2:mem:productosdb
7 spring.datasource.driverClassName=org.h2.Driver
8 spring.datasource.username=sa
9 spring.datasource.password=
10
11 spring.h2.console.enabled=true
12 spring.h2.console.path=/h2-console
13
14 spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
15 spring.jpa.hibernate.ddl-auto=create
16
17 eureka.client.service-url.defaultZone=http://localhost:8761/eureka
18 feign.hystrix.enabled=true
```

The status bar at the bottom indicates 'Ln 18, Col 27', 'Spaces: 4', 'UTF-8', 'CRLF', and 'Java Properties'.

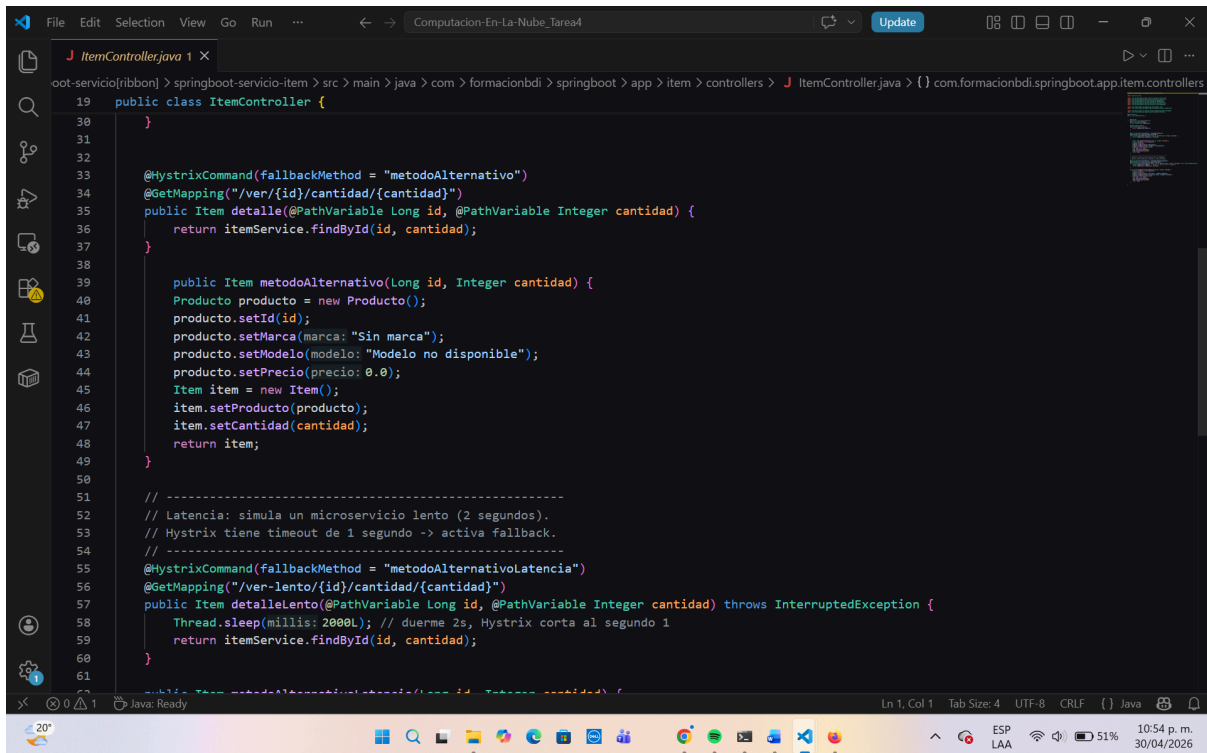


The screenshot shows an IDE window titled 'Computation-En-La-Nube_Tarea4'. The editor displays the 'application.properties' file for the project 'springboot-servicio-zuul-server'. The properties are as follows:

```
1 spring.application.name=servicio-zuul-server
2 server.port=8090
3
4 eureka.client.service-url.defaultZone=http://localhost:8761/eureka
5
6 # Ruteo dinamico
7 zuul.routes.items.service-id=servicio-items
8 zuul.routes.items.path=/api/items/**
9
10 zuul.routes.productos.service-id=servicio-productos
11 zuul.routes.productos.path=/api/productos/**
12
13 # Balanceo con Ribbon (dos instancias de servicio-items)
14 servicio-items.ribbon.listOfServers=localhost:8002,localhost:8003
15
16 # Hystrix timeout: 1 segundo
17 hystrix.command.default.execution.isolation.thread.timeoutInMilliseconds=1000
18 hystrix.command.default.execution.timeout.enabled=true
19
20 ribbon.ConnectTimeout=1000
21 ribbon.ReadTimeout=1000
22
```

The status bar at the bottom indicates 'Ln 22, Col 1', 'Spaces: 4', 'UTF-8', 'LF', and 'Java Properties'.

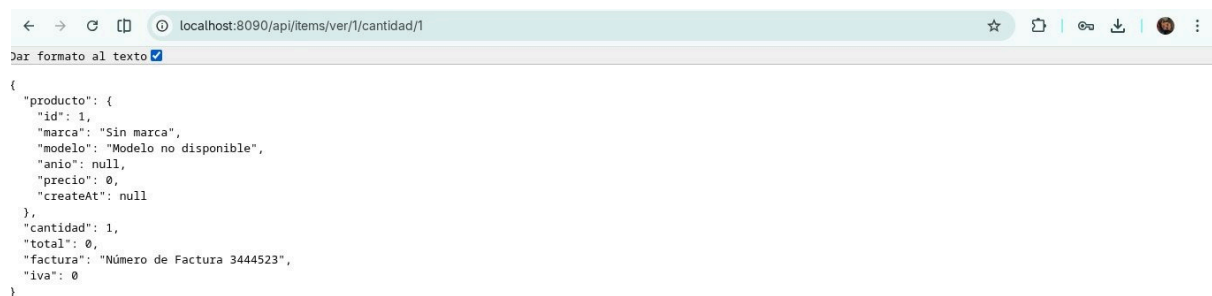
- Recuperación de errores cuando al llamar un microservicio, éste falla. Llamar a un método alternativo mediante la anotación: `@HystrixCommand`



```
19 public class ItemController {
20 }
21
22 @HystrixCommand(fallbackMethod = "metodoAlternativo")
23 @GetMapping("/ver/{id}/cantidad/{cantidad}")
24 public Item detalle(@PathVariable Long id, @PathVariable Integer cantidad) {
25     return itemService.findById(id, cantidad);
26 }
27
28 public Item metodoAlternativo(Long id, Integer cantidad) {
29     Producto producto = new Producto();
30     producto.setId(id);
31     producto.setMarca(marca: "Sin marca");
32     producto.setModelo(modelo: "Modelo no disponible");
33     producto.setPrecio(precio: 0.0);
34     Item item = new Item();
35     item.setProducto(producto);
36     item.setCantidad(cantidad);
37     return item;
38 }
39
40 // -----
41 // Latencia: simula un microservicio lento (2 segundos).
42 // Hystrix tiene timeout de 1 segundo -> activa fallback.
43 // -----
44 @HystrixCommand(fallbackMethod = "metodoAlternativoLatencia")
45 @GetMapping("/ver-lento/{id}/cantidad/{cantidad}")
46 public Item detalleLento(@PathVariable Long id, @PathVariable Integer cantidad) throws InterruptedException {
47     Thread.sleep(millis: 2000L); // duerme 2s, Hystrix corta al segundo 1
48     return itemService.findById(id, cantidad);
49 }
50
51 public Item metodoAlternativoLatencia(Long id, Integer cantidad) {
52     return item;
53 }
```

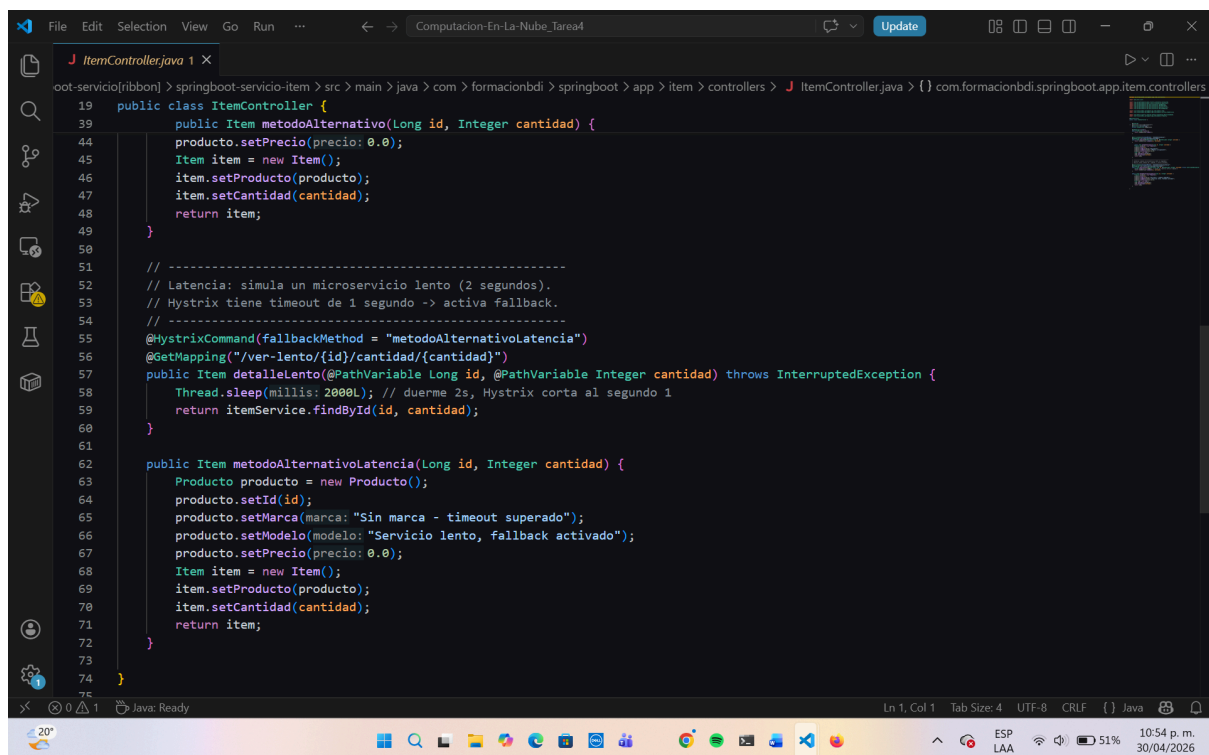
URL: <http://localhost:8090/api/items/ver/1/cantidad/1>

Para demostrar la recuperación de errores, se detuvo el servicio-productos. Hystrix detectó el fallo y activó automáticamente el metodoAlternativo(), devolviendo una respuesta controlada con valores predeterminados ("Sin marca", "Modelo no disponible") en lugar de un error 500.



```
{
  "producto": {
    "id": 1,
    "marca": "Sin marca",
    "modelo": "Modelo no disponible",
    "anio": null,
    "precio": 0,
    "createAt": null
  },
  "cantidad": 1,
  "total": 0,
  "factura": "Número de Factura 3444523",
  "iva": 0
}
```

- Latencia. Recuperación de errores cuando al llamar un microservicio, este falla porque se tardó más de un segundo.



```
19 public class ItemController {
39     public Item metodoAlternativo(Long id, Integer cantidad) {
44         producto.setPrecio(precio: 0.0);
45         Item item = new Item();
46         item.setProducto(producto);
47         item.setCantidad(cantidad);
48         return item;
49     }
50
51     // -----
52     // Latencia: simula un microservicio lento (2 segundos).
53     // Hystrix tiene timeout de 1 segundo -> activa fallback.
54     // -----
55     @HystrixCommand(fallbackMethod = "metodoAlternativoLatencia")
56     @GetMapping("/ver-lento/{id}/cantidad/{cantidad}")
57     public Item detalleLento(@PathVariable Long id, @PathVariable Integer cantidad) throws InterruptedException {
58         Thread.sleep(millis: 2000L); // duerme 2s, Hystrix corta al segundo 1
59         return itemService.findById(id, cantidad);
60     }
61
62     public Item metodoAlternativoLatencia(Long id, Integer cantidad) {
63         Producto producto = new Producto();
64         producto.setId(id);
65         producto.setMarca(marca: "Sin marca - timeout superado");
66         producto.setModelo(modelo: "Servicio lento, fallback activado");
67         producto.setPrecio(precio: 0.0);
68         Item item = new Item();
69         item.setProducto(producto);
70         item.setCantidad(cantidad);
71         return item;
72     }
73
74 }
75 }
```

URL: <http://localhost:8090/api/items/ver-lento/1/cantidad/1>

La respuesta muestra que Zuul detectó la latencia excesiva y activó el fallback del FallbackProvider, devolviendo "Sin marca - timeout Zuul" con precio 0 en lugar de esperar indefinidamente o lanzar un error 500.



```
{
  "producto": {
    "id": 0,
    "marca": "Sin marca - timeout Zuul",
    "modelo": "Servicio lento, fallback de Zuul activado",
    "precio": 0
  },
  "cantidad": 1,
  "total": 0
}
```

Conclusión

Se implementó exitosamente el servidor Zuul de Netflix como API Gateway en una arquitectura de microservicios con Spring Cloud. Las cuatro funcionalidades requeridas fueron demostradas:

1. Ruteo dinámico: Zuul redirige peticiones a servicio-productos y servicio-items mediante rutas configuradas con nombres de Eureka.
2. Balanceo con Hystrix: Ribbon balancea entre instancias disponibles y Hystrix actúa como circuit breaker.
3. Recuperación de errores (@HystrixCommand): Al fallar el microservicio, se ejecuta automáticamente el método alternativo devolviendo datos predeterminados.
4. Latencia: El FallbackProvider de Zuul intercepta timeouts y devuelve una respuesta controlada cuando el microservicio tarda más de 1 segundo.